

Report

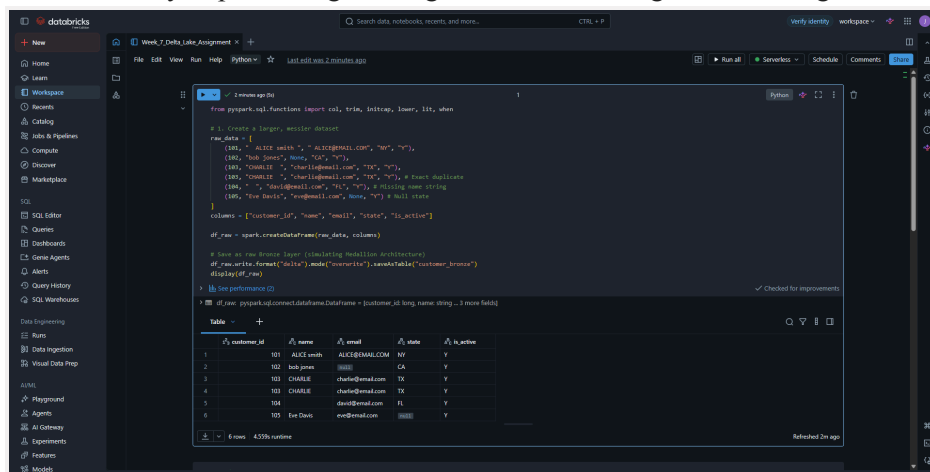
Objective:

To design and implement a robust data ingestion and cleaning pipeline using Apache Spark, culminating in a Delta Lake MERGE operation to handle incremental data updates.

1. Data Loading (Bronze Layer)

To begin the pipeline, a raw, unstructured dataset was generated to represent incoming source records. This dataset intentionally included common data quality issues such as leading and trailing whitespaces, inconsistent capitalization, missing string values, nulls, and duplicate records.

The dataset was saved into the Databricks default Delta Lake format. Physically, a Delta table consists of underlying Parquet files paired with a `_delta_log` directory of JSON files. This transaction log enables ACID transactions and reliable concurrent writes, which a plain Parquet or Hive table cannot support. In the context of a Medallion Architecture, this raw, unaltered data acts as the Bronze layer, preserving the original state for lineage and auditing.



```
from pyspark.sql.functions import col, trim, lower, lower, lit, when

# 1. Create a larger, messier dataset
raw_data = [
    (lit, " Alice Smith ", " ALICE@GMAIL.COM ", "NY", "Y"),
    (lit, "Bob Jones", "None", "CA", "Y"),
    (lit, "Charlie ", "charlie@gmail.com", "TX", "Y"),
    (lit, "Charlie ", "charlie@gmail.com", "TX", "Y"), # Intentional duplicate
    (lit, " ", "dave@gmail.com", "FL", "Y"), # Missing name string
    (lit, "Eve Davis", "eve@gmail.com", "None", "Y") # Null state
]

columns = ["customer_id", "name", "email", "state", "is_active"]

df_raw = spark.createDataFrame(raw_data, columns)

# Now we're ready to load (creating the Delta Lake architecture)
df_raw.write.format("delta").mode("overwrite").saveAsTable("customer_bronze")
display(df_raw)

# See performance (2)
```

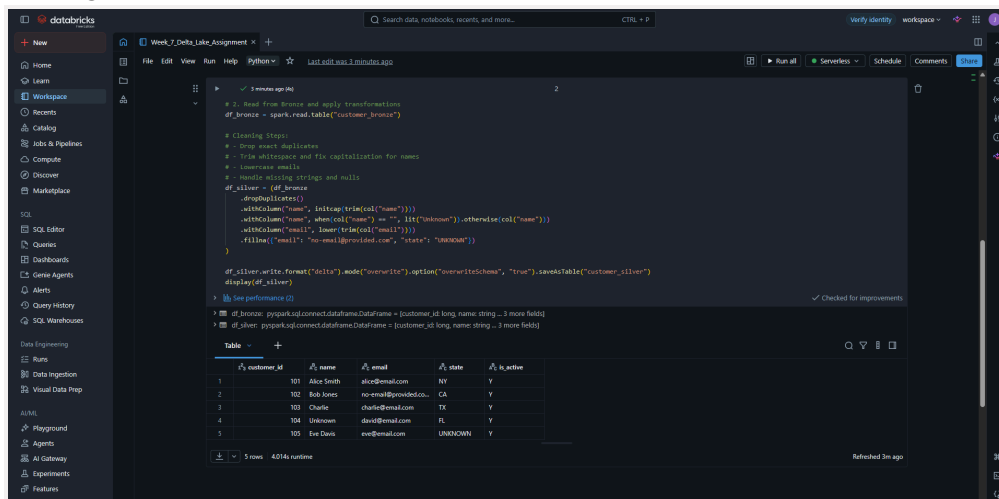
customer_id	name	email	state	is_active
1	Alice Smith	ALICE@GMAIL.COM	NY	Y
2	Bob Jones		CA	Y
3	Charlie	charlie@gmail.com	TX	Y
4	Charlie	charlie@gmail.com	TX	Y
5		dave@gmail.com	FL	Y
6	Eve Davis	eve@gmail.com		Y

2. Data Cleaning & Structuring (Silver Layer)

Following the multi-hop architecture pattern, the data was progressively refined from the Bronze layer into a cleaner, enterprise-wide Silver layer.

The following PySpark transformations were applied to structure the data:

- **Deduplication:** Dropped identical rows to ensure data integrity.
- **String Manipulation:** Applied `trim()` and `initcap()` to standardize names, and `lower()` to standardize email addresses.
- **Null Handling:** Replaced empty strings with a default "Unknown" string and used `.fillna()` to provide default values for missing emails and states.



```
# 2. Read from Bronze and apply transformations
df_bronze = spark.read.table("customer_bronze")

# Cleaning Steps
# - Drop exact duplicates
# - Trim whitespace and fix capitalization for names
# - Increase email
# - Handle missing strings and nulls
df_silver = (df_bronze
    .dropDuplicates()
    .withColumn("name", initcap(trim(col("name"))))
    .withColumn("name", when(col("name") == "", lit("Unknown")).otherwise(col("name")))
    .withColumn("email", lower(trim(col("email"))))
    .fillna("email", "no-email@provided.com", "state", "UNKNOWN")
)

df_silver.write.format("delta").mode("overwrite").options("overwriteSchema", "true").saveAsTable("customer_silver")
display(df_silver)

# See performance (2)
```

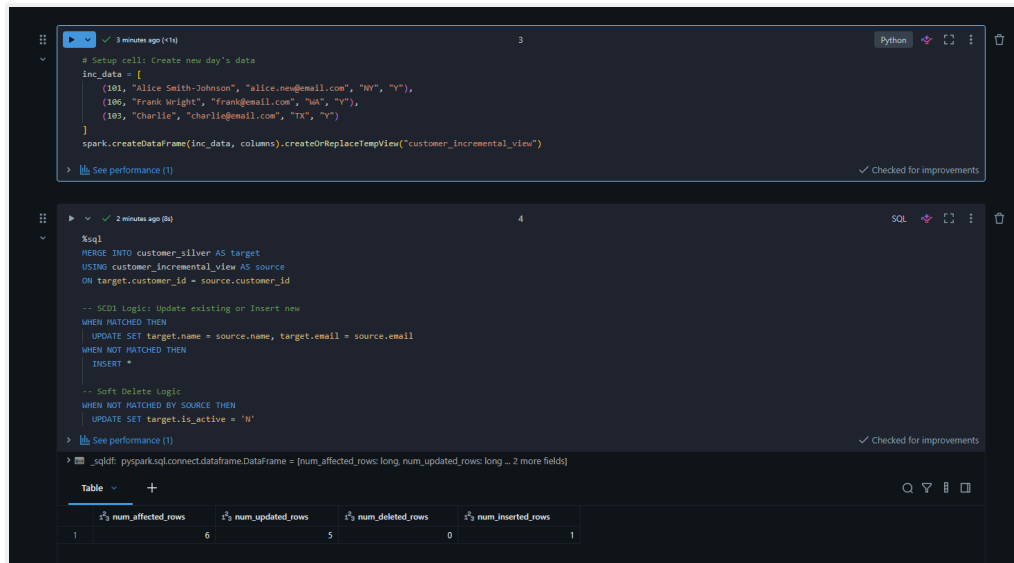
customer_id	name	email	state	is_active
1	Alice Smith	alice@gmail.com	NY	Y
2	Bob Jones	no-email@provided.com	CA	Y
3	Charlie	charlie@gmail.com	TX	Y
4	Unknown	dave@gmail.com	FL	Y
5	Eve Davis	eve@gmail.com	UNKNOWN	Y

3. Incremental Upserts (SCD1) & The MERGE Operation

To test the pipeline's ability to handle evolving data over time, a second dataset (**customer_incremental**) was introduced representing a new day's data.

A SQL **MERGE** statement was used to upsert this incremental data into the target Silver table. The operation relied on up to three distinct matching clauses to determine the outcome of each row:

- **WHEN MATCHED:** Implemented Slowly Changing Dimension Type 1 (SCD1) logic by updating the target row's email and name with the new values from the source, intentionally overwriting the old state with no history kept.
- **WHEN NOT MATCHED:** Inserted entirely new customer records that only existed in the source data.



```
# Setup cell: Create new day's data
inc_data = [
    (101, "Alice Smith-Johnson", "alice.new@email.com", "NY", "Y"),
    (106, "Frank Wright", "frank@email.com", "MA", "Y"),
    (103, "Charlie", "charlie@email.com", "TX", "Y")
]

spark.createDataFrame(inc_data, columns).createOrReplaceTempView("customer_incremental_view")

-- See performance (1)

%sql
MERGE INTO customer_silver AS target
USING customer_incremental_view AS source
ON target.customer_id = source.customer_id

-- SCD1 logic: Update existing or Insert new
WHEN MATCHED THEN
  UPDATE SET target.name = source.name, target.email = source.email
WHEN NOT MATCHED THEN
  INSERT *

-- Soft Delete Logic
WHEN NOT MATCHED BY SOURCE THEN
  UPDATE SET target.is_active = 'N'
```

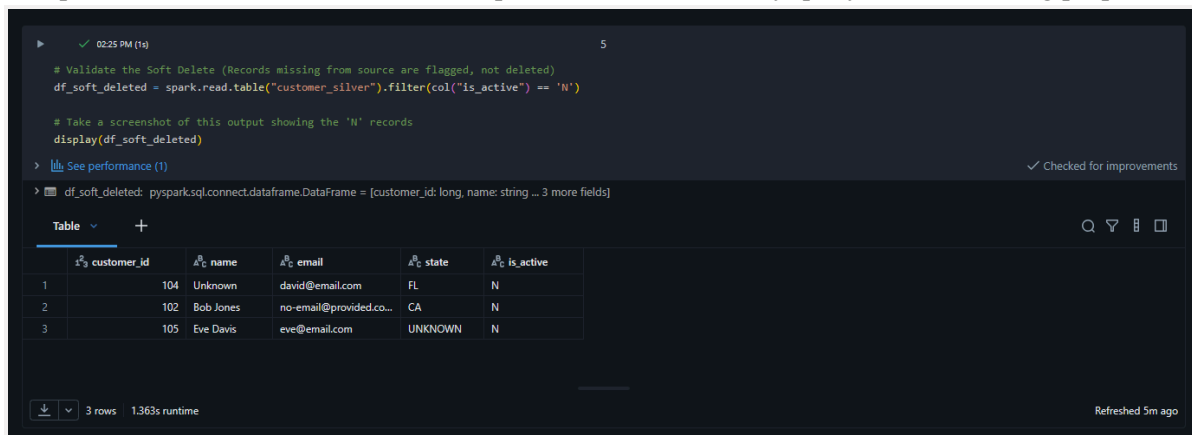
	num_affected_rows	num_updated_rows	num_deleted_rows	num_inserted_rows
1	6	5	0	1

4. Historical Preservation via Soft Deletes

Instead of executing a hard **DELETE** for records that dropped out of the source system, the **MERGE** operation utilized the third matching clause.

- **WHEN NOT MATCHED BY SOURCE:** Handled keys that were present in the target but absent from the source.

This clause was used to implement a soft delete by updating an **is_active** flag to 'N' instead of physically erasing the row. This approach preserves historical data and ensures past records remain fully queryable for auditing purposes.



```
# Validate the Soft Delete (Records missing from source are flagged, not deleted)
df_soft_deleted = spark.read.table("customer_silver").filter(col("is_active") == 'N')

# Take a screenshot of this output showing the 'N' records
display(df_soft_deleted)

-- See performance (1)

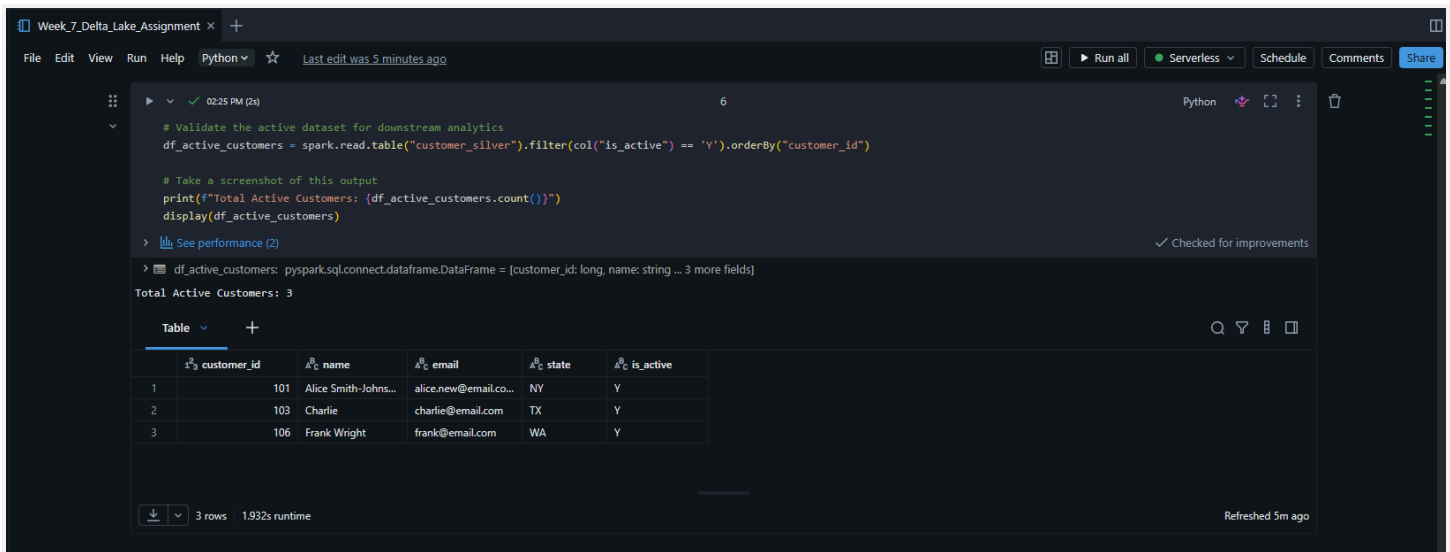
df_soft_deleted: pyspark.sql.connect.dataframe.DataFrame = [customer_id: long, name: string ... 3 more fields]
```

	customer_id	name	email	state	is_active
1	104	Unknown	david@email.com	FL	N
2	102	Bob Jones	no-email@provided.co...	CA	N
3	105	Eve Davis	eve@email.com	UNKNOWN	N

3 rows 1.363s runtime Refreshed 5m ago

5. Validation and Final Output

To confirm the success of the pipeline, the dataset was queried to validate the downstream impact. The current, active state of the table can easily be accessed by analysts simply by filtering for `is_active = 'Y'`.



The screenshot shows a Databricks notebook interface with a Python cell. The code filters the 'customer_silver' table for active customers (is_active = 'Y') and displays the result. The output shows 3 rows of data.

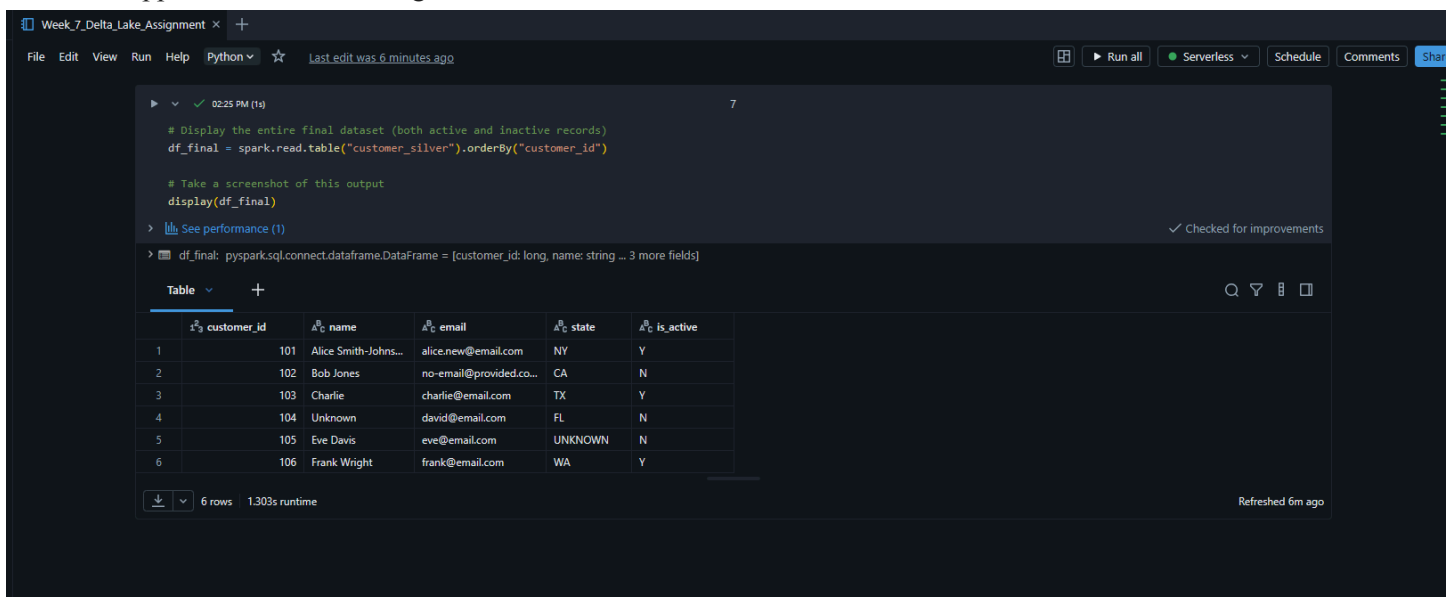
```
# Validate the active dataset for downstream analytics
df_active_customers = spark.read.table("customer_silver").filter(col("is_active") == 'Y').orderBy("customer_id")

# Take a screenshot of this output
print(f"Total Active Customers: {df_active_customers.count()}")
display(df_active_customers)
```

Total Active Customers: 3

customer_id	name	email	state	is_active
101	Alice Smith-Johns...	alice.new@email.co...	NY	Y
103	Charlie	charlie@email.com	TX	Y
106	Frank Wright	frank@email.com	WA	Y

The final dataset accurately reflects the updated information for existing keys, the insertion of the new keys, and the successful application of the 'N' flag for stale records.



The screenshot shows a Databricks notebook interface with a Python cell. The code displays the entire final dataset (both active and inactive records) from the 'customer_silver' table. The output shows 6 rows of data.

```
# Display the entire final dataset (both active and inactive records)
df_final = spark.read.table("customer_silver").orderBy("customer_id")

# Take a screenshot of this output
display(df_final)
```

df_final: pyspark.sql.connect.dataframe.DataFrame = [customer_id: long, name: string ... 3 more fields]

customer_id	name	email	state	is_active
101	Alice Smith-Johns...	alice.new@email.com	NY	Y
102	Bob Jones	no-email@provided.co...	CA	N
103	Charlie	charlie@email.com	TX	Y
104	Unknown	david@email.com	FL	N
105	Eve Davis	eve@email.com	UNKNOWN	N
106	Frank Wright	frank@email.com	WA	Y